

Module 8 Lab Session 1: Shell and Signals

Module M8 Angular 22 Fundamentals

Session 1

Before you begin

- Confirm **Node 20+** and the **Angular CLI** install.

Checkpoint Tier 1 evidence path

When you finish Exercise 1 below, you should be able to show:

- A working **ng serve** loop on `tms-client`.
- **app.config.ts** configured for the programme baseline (including zone-friendly change detection as specified in the steps).
- Dashboard UI driven by **signal()** and **computed()** clicking or interacting updates the screen without stale template reads.

You are building the **TMS client** the browser application that students, instructors, and admins will actually use. Every Exercise in this workbook adds a layer to the same project. By the end, you will have a working Angular application that talks to the .NET API you built in M6 and M7.

Ground rules for this workbook:

- Every component is standalone. There is no `app.module.ts`.
- Use `input()`, `output()`, `signal()`, `computed()`. Not the decorator versions.
- Use `@if` and `@for`. Not `*ngIf` or `*ngFor`.
- Use Reactive Forms (`FormGroup`). Not `[(ngModel)]` on complex forms.
- Work in short cycles: write code, save, check the browser, fix, repeat. Do not write 50 lines before checking if it works.

Shared Type (Create Once, Reuse Everywhere)

Before starting Exercise 1, create the file `src/app/models/course.model.ts`:

```

/**
 * List row from the TMS API – mirrors `CourseResponseDto` on `GET /api
 /courses`.
 * ASP.NET Core defaults to camelCase JSON (`id`, `maxCapacity`, ...).
 */
export interface Course {
  id: number;
  code: string;
  title: string;
  maxCapacity: number;
  enrollmentCount: number;
}

/** Envelope for `GET /api/courses` – TMS API contract List shape (`Pag
edResponse<T>`). */
export interface PagedResponse<T> {
  items: T[];
  totalCount: number;
  page: number;
  pageSize: number;
  totalPages: number;
  hasPrevious: boolean;
  hasNext: boolean;
}

/** One link from `CourseDetailDto.Links` on `GET /api/courses/{id}`. *
 /
export interface CourseLink {
  href: string;
  rel: string;
  method: string;
}

/** Detail payload – mirrors `CourseDetailDto` (list rows do not includ
e `links`). */
export interface CourseDetail extends Course {
  links: readonly CourseLink[];
}

```

You will use `Course` in every Exercise; `PagedResponse`, `CourseDetail`, and `CourseLink` matter from Exercise 6 onward (and any time you call `getById`).

Exercise 1 Setting Up the Project and Your First Signals

Liya is a student at CoTBE. She opens the TMS in her browser expecting to see her dashboard: her name, how many credits she has earned, and whether she is eligible for graduation. Right now, there is nothing. You are going to build that dashboard.

Step 1: Create the Angular Project

Open a terminal. Navigate to where you want the project to live. Here is the command you will run but before you type it, understand what each piece means:

```
ng new tms-client --style=scss --ssr=false --standalone --routing
```

Flag	What It Does	Why We Use It
tms-client	The name of the project folder that gets created	This is our Training Management System frontend
--style=scss	Sets the stylesheet format to SCSS instead of plain CSS	SCSS lets you nest styles and use variables cleaner than raw CSS for a real project
--ssr=false	Disables Server-Side Rendering	SSR is for public websites that need search engine indexing. The TMS is a login-protected app we do not need it, and it adds complexity we do not want right now
--standalone	Generates the project without the legacy NgModule system	Modern Angular uses standalone components by default. This flag ensures the CLI does not generate app.module.ts
--routing	Sets up an app.routes.ts file and <router-outlet>	We need URL routing from the start (/dashboard, /courses/:id, etc.)

Now run the command. The CLI may ask a few follow-up questions accept the defaults. When it finishes, move into the project:

```
cd tms-client
```

Code .

]Step 2: Configure the Application Shell

Open src/app/app.config.ts. The CLI generated some of this already, but you need to add a few things. Here is the complete file replace what is there with this:

```
import { ApplicationConfig, provideZonelessChangeDetection } from "@angular/core";
import { provideRouter, withComponentInputBinding } from "@angular/rout
```

```

er";
import { provideHttpClient } from "@angular/common/http";
import { routes } from "../app.routes";

export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(),
  ],
};

```

This file is the entry point for Angular's global configuration. Each `provideX()` function registers a capability:

Provider	What It Registers	What Breaks Without It
<code>provideZoneChangeDetection(...)</code>	Controls when Angular checks the screen for updates. The <code>eventCoalescing: true</code> option batches multiple rapid events into a single check.	Angular cannot detect any changes the screen never updates
<code>provideRouter(routes, withComponentInputBinding())</code>	Connects URLs to components. The <code>withComponentInputBinding()</code> part is a shortcut we will use in Exercise 4 it lets URL parameters flow directly into component inputs.	Navigating to any URL shows a blank page
<code>provideHttpClient()</code>	Gives Angular the ability to make HTTP requests to your .NET API	Any service using <code>HttpClient</code> crashes with <code>NullInjectorError</code>

We are adding `provideHttpClient()` now even though you will not use it until Exercise 6. It costs nothing to add it early, and forgetting it later produces a confusing error message.

Step 3: Generate the Dashboard Component

```
ng generate component features/student-dashboard --type=component
```

This creates a folder at `src/app/features/student-dashboard/` with four files:

File	Purpose
<code>student-dashboard.component.ts</code>	The TypeScript class your component's logic and data
<code>student-dashboard.component.html</code>	The HTML template what the user sees
<code>student-dashboard.component.scss</code>	The styles how it looks
<code>student-dashboard.component.spec.ts</code>	The test file you will use this in M11

Architecture Note: File Suffix Conventions in Modern Angular Modern Angular CLI tools default to suffix-less file generation (e.g., `student-dashboard.ts` instead of `student-dashboard.component.ts`). In this Training Management System (TMS) codebase, we explicitly retain `.component.ts`, `.service.ts`, and `.model.ts` type suffixes to prevent naming collisions when colocating domain features. If your CLI environment defaults to suffix-less generation, pass `--type=component` when generating (ng generate component features/student-dashboard --type=component) or add "type": "component" under `@schematics/angular:component` in your `angular.json`.

Step 4: Build the Signal State

Open `student-dashboard.component.ts`. Replace the generated content with the code below. Read the comments every line is explained:

```
// These are the Angular functions we need. signal() and computed() come from Angular's core.
import { Component, signal, computed } from "@angular/core";

// The @Component decorator tells Angular: "This class is a visual component."
// It is metadata it describes how this class connects to the HTML template.
@Component({
  selector: "app-student-dashboard", // The HTML tag name: <app-student-dashboard />
  standalone: true, // This component manages its own imports (no NgModule)
  templateUrl: "./student-dashboard.component.html", // Points to the HTML file
  styleUrls: ["./student-dashboard.component.scss"], // Points to the styles file
})
```

```

export class StudentDashboardComponent {
  // signal('Liya Kebede') creates a reactive variable. Angular watches
  // it.
  // When its value changes, Angular automatically updates the part of
  // the screen that displays it.
  studentName = signal("Liya Kebede");
  earnedCredits = signal(45);

  // computed() creates a read-only signal that derives its value from
  // other signals.
  // It recalculates automatically whenever earnedCredits() changes no
  // manual refresh.
  graduationStatus = computed(() =>
    this.earnedCredits() >= 120 ? "Eligible for Graduation" : "In Progr
    ess",
  );

  // A regular method. When called, it updates the earnedCredits signal.
  // The .update() method receives the current value (c) and returns th
  // e new value (c + 3).
  registerForClass() {
    this.earnedCredits.update((c) => c + 3);
  }
}

```

Step 5: Build the Template

Open student-dashboard.component.html. Delete whatever the CLI generated and paste this:

```

<div class="dashboard">
  <h1>Welcome, {{ studentName() }}</h1>
  <p>Credits Earned: {{ earnedCredits() }}</p>
  <p>Graduation Status: {{ graduationStatus() }}</p>
  <button (click)="registerForClass()">
    Register for a Class (+3 credits)
  </button>
</div>

```

A few things to understand about this template:

- `{{ studentName() }}` The double curly braces (`{{ }}`) are Angular's way of displaying a value from the TypeScript class. The parentheses (`()`) are required because `studentName` is a signal you are calling it to get the current value inside.
- `(click)="registerForClass()"` The parentheses around `click` mean this is an **event binding**. When the user clicks the button, Angular calls the `registerForClass()` method in your TypeScript class. This is Angular's

equivalent of `onClick` in plain HTML, but it is wired directly to your component.

Step 6: Add a Route

Open `src/app/app.routes.ts`. This file tells Angular which component to display for each URL:

```
import { Routes } from "@angular/router";

export const routes: Routes = [
  {
    path: "dashboard",
    loadComponent: () =>
      import("../features/student-dashboard/student-dashboard.component")
        .then(
          (m) => m.StudentDashboardComponent,
        ),
  },
  { path: "", redirectTo: "dashboard", pathMatch: "full" },
];
```

Let's break this down:

- **path: 'dashboard'** When the browser URL is `/dashboard`, load this component.
- **loadComponent: () => import(...).then(m => m.StudentDashboardComponent)** This is called **lazy loading**. Instead of loading the dashboard code when the app starts, Angular loads it only when the user actually visits `/dashboard`. The `.then(m => m.StudentDashboardComponent)` part extracts the specific class from the imported file. This pattern looks complex, but you will copy it for every route the only things that change are the file path and the component name.
- **{ path: '', redirectTo: 'dashboard', pathMatch: 'full' }** If someone visits the root URL (`/`), automatically send them to `/dashboard`.

Finally, open `src/app/app.component.html`. Delete everything the CLI generated (the Angular welcome page with the logo). Replace it with just:

```
<router-outlet />
```

The `<router-outlet>` is a placeholder. It tells Angular: "Render the current route's component here." Without it, your routes are configured but there is nowhere to display them you get a blank page.

Step 7: Run It

`ng serve`

Open `http://localhost:4200/dashboard` in your browser.

What you should see: Liya's name, her credits (45), and her graduation status (In Progress). Click the button. The credits go to 48, then 51, and so on. The graduation status stays "In Progress" until credits reach 120, then it flips to "Eligible for Graduation."

Troubleshooting

Problem	Likely Cause	Fix
Blank page at /dashboard	Missing <code><router-outlet /></code> in <code>app.component.html</code>	Add <code><router-outlet /></code>
<code>{{ earnedCredits }}</code> shows [object Object]	Missing parentheses on the signal	Change to <code>{{ earnedCredits() }}</code>
"Cannot find module" error	Wrong import path after generating	Check the relative path in <code>app.routes.ts</code> matches the actual file location
Port 4200 already in use	Another process is using it	Run <code>ng serve --port 4300</code> or kill the other process

Checkpoint 1

Before moving to Exercise 2, confirm:

- ☐ `ng serve` starts without errors
- ☐ The dashboard renders with dynamic signal data
- ☐ Clicking the button updates both the credits number and the graduation status text